



JavaScript -- JS

- 1 JS概述
- 2 JS语法规则
- 3 BOM与DOM对象
- 4 JS事件

1

JS概述

2

JS语法规则

3

BOM与DOM对象

4

JS事件

- 什么是JS？有什么作用？

- Html-----页面的结构-----人的面部（素颜）
- CSS-----美化页面-----化妆
- JavaScript(JS)-----控制页面特效----人的大脑、灵魂
- JavaScript是强大的Web**脚本语言**；

编程语言：独立编写，独立运行。如：C语言、Java

脚本语言：无法独立运行，必须嵌入到其它语言中，配合使用；
可以被浏览器直接解析运行。

- JavaScript的作用：控制页面特效。

如：对Html元素进行动态控制；表单校验；样式动态调整等。

- JS入门案例
 - 步骤1：创建一个Html文档
 - 步骤2：在Html文档引入<script>标签
 - 步骤3：在<script>标签编写JS代码

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title></title>
    <script type="text/javascript">
      //int a = 10, b = 20;
      var a = 10, b = 20;

      // int sum = a + b;
      var sum = a + b;

      //System.out.println(sum);
      Console.log(sum);
      alert(sum);
    </script>
  </head>
  <body>
  </body>
</html>
```

- JS的语言特征及其组成

- 语言特征

- (1) 一种解释型语言，无需编译，由浏览器边解释、边执行；
- (2) 必须嵌入到Html代码中运行，由上到下依次执行；
- (3) 一种弱类型的语言，没有明确的变量类型之分；
- (4) JavaScript与Java没有任何关系；
- (5) 有很多JS框架，方便前端程序员进行开发，如：JQuery, Dojo.

- JS组成

- (1) ECMAScript（核心）：规定了JS的语法与基本对象；
- (2) BOM，浏览器对象模型：与浏览器交互的方法和接口；
- (3) DOM，文档对象模型：处理网页内容的方法和接口。

注：Html标签都被封装到了DOM对象中

- JS的引入方式（内部脚本、外部引入）

- 内部脚本

- （1）在当前Html页面中添加<script>标签，<script>内部即可书写JS代码；

- （2）格式：<script type="text/javascript">

- JS代码

- </script>

- （3）示例：

```
<script type="text/javascript">  
    //JavaScript代码  
</script>
```

JS概述

- JS的引入方式（内部脚本、外部引入）

- 外部引入（解决一段JS代码需要被多个Html使用）

步骤：(1)新建一个js文件，编写JS代码；

(2)将这个js文件引入到html文档中；

如何引入？使用<script>标签

格式：<script type = “text/javascript” src = “js文件路径”/></script>

注：(1)文件路径要写相对路径；

(2)外部引入了js文件时，这对<script>标签内部禁止写入其它的JS代码，即使写了也不会执行。

```
<script type="text/javascript" src="demo.js">  
    //这部分代码不会被执行，因为被引入的demo.js代码覆盖  
</script>
```

- <script>标签规范的放置位置
 - 规范放置位置：在<body>的结束标签之前

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <h1>欢迎</h1>
    <script></script>
  </body>
</html>
```

优点：保证Html展示内容优先加载，再加载脚本，增强用户体验。

学习时可以放置在其它位置，如在<head>标签中。

- 1 JS概述
- 2 **JS语法规则**
- 3 BOM与DOM对象
- 4 JS事件

JS语法规则

- JS的注释（与一般编程语言一致）

- 单行注释

//Content

- 多行注释

/*Content*/

```
<script type="text/javascript">  
    //单行注释  
    /*  
        多行注释  
        多行注释  
    */  
</script>
```

- JS的变量

- 变量：标识内存中的一块空间，用来存储数据，且值是可变的。

- 变量的声明：

- var 变量名; //默认是undefined

- 变量的声明与赋值：

- var 变量名 = 值;

- JS变量的命名规范：

- (1)必须以字母或下划线开头，中间可以是数字、字符或下划线；

- (2)不能包含空格等特殊符号；

- (3)不能使用JS的关键字作为变量名，如：var, function;

- (4)JS严格区分大小写。

JS语法规则

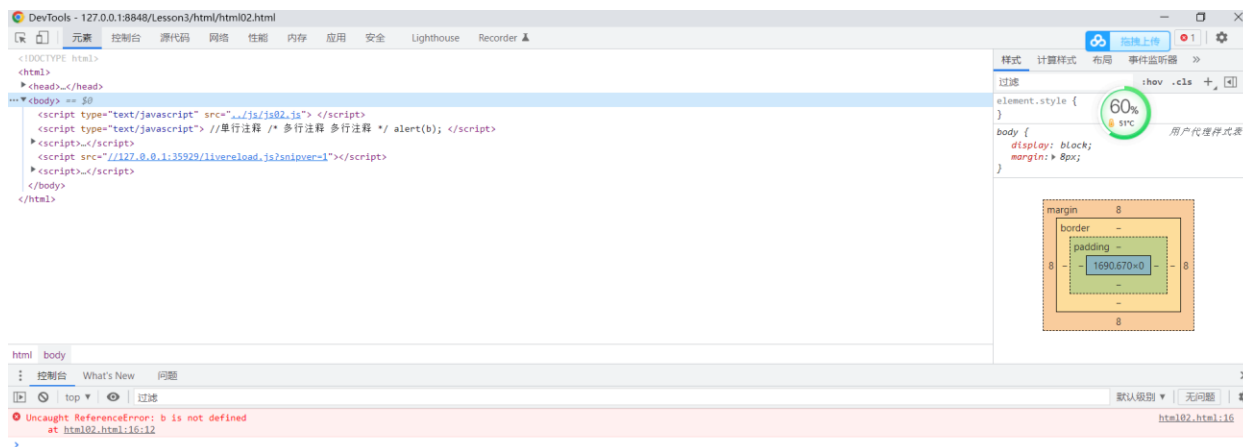
- JS的错误调试

- 打开浏览器：按F12---开发者工具；（部分电脑Fn+F12）

- (1)元素：查看所有的前端代码：包括了Html元素，CSS样式，JS脚本；

- (2)控制台：查看代码错误和输出（Console.log()）；

- (3)源代码：显示了Html元素与JS脚本，可以进行调试。



- JS变量的基本数据类型

- JS中的基本数据类型（5种）

- (1)string, 字符串类型, ""和''都是字符串, Javascript没有单个字符;

- (2)boolean, 布尔类型, 取值true或false;

- (3)number, 数值类型, 取值为任意的数字;

- (4)null, 空, 一个占位符;

- (5)undefined, 未定义类型, 表示变量声明但未赋予具体的值。

- 注: 1. undefined是从null派生而来的, 故undefined 等价于 null;

- 2. JS是弱类型语言, 变量使用不用严格遵循规范, 一个变量可以被赋予任意类型的值, 后赋的值取代之前赋的值;

- 3. 可以利用typeof()去分辨变量的类型。例外: typeof(null);

- JS变量的引用数据类型
 - JS中的引用数据类型都是对象（Object）
 - JS中对象的创建方式：

```
var strObj = new String();
```

```
var strObj = new String;
```

```
<head>
  <meta charset="utf-8">
  <title></title>
  <script type="text/javascript">
    var str = new String();
    alert(typeof(str));
  </script>
</head>
```

- JS的比较运算符

随堂练习：

```
var x=8;
alert(x==8);
alert(x===8);

alert(x=="8");
alert(x==="8");
```

== 逻辑等。仅仅对比 值。

=== 全等。对比值并且对比类型。

如果值和类型都相同，则为 true；值和类型有一个不同，则为 false。

- JS的逻辑运算符

运算符	描述	例子
&&	and	(x < 10 && y > 1) 为 true
	or	(x==5 y==5) 为 false
!	not	!(x==y) 为 true

注：JS的逻辑运算符没有 &，|。

- JS的正则对象（RegExp）

- JS中正则对象的创建（利用正则表达式时创建的对象）
- 正则表达式通常被用来检索、替换那些符合某个模式(规则)的文本。
- 适用场景：判断用户名、密码设置是否符合规则

(1)方式一： `var regObj = /^表达式$/`;（直接量方式，常用）

(2)方式二： `var regObj = /表达式/`;（普通方式，没有边界符）

注：直接量方式存在边界，^开始，\$结束。

reg赋值的是一个正则对象，而不是字符串，不要加””。

- 正则对象的方法

`boolean regObj.test(string)`; 判断字符串是否符合正则表达式的规则。

- JS的正则对象（RegExp）

- 直接量方式 vs 普通方式

(1)直接量方式：只要有不符合规则的字符存在，返回false，全部符合返回true；适用于表单校验。

(2)普通方式：只要有符合规则的字符存在，返回true，全部不符合返回false；适用于字符串的查找、替换。

```
<script type="text/javascript">
  var regObj_1 = /^s+$/ //表示1个或多个空格
  console.log(regObj_1.test("  "));
  console.log(regObj_1.test(" a "));
  var regObj_2 = /\s+/ //表示1个或多个空格
  console.log(regObj_2.test("  "));
  console.log(regObj_2.test(" a "));
  console.log(regObj_2.test("aaa"));
</script>
```

- JS的数组对象

- JS数组的特性（相当于java中的ArrayList）

- (1)数组中的成员没有类型限制，可以存放任意类型；

- (2)数组的长度可以自动修改

- JS数组的4种创建方式（相当于java中的ArrayList）

- (1)var arr = [1,2,3];

- (2)var arr = new Array(); //数组的长度默认为0;

- (3)var arr = new Array(4); //数组的长度为4，每个元素的类型是undefined;
[undefined, undefined, undefined, undefined]

- (4)var arr = new Array(1,4); //数组的长度为2，两个元素： 1,4。

JS语法规则

- JS的数组对象

- JS数组的常用属性和方法

(1)属性：length，表示数组的长度；

```
<script type="text/javascript">
  var arr = ["hello",1,4,"haha"];
  alert(arr.length);
  arr[1] = 3;
  alert(arr.length);
  arr[4] = "互联网金融";
  alert(arr.length);
  arr[10] = "Anhui University";
  alert(arr.length);
</script>
```

JS的数组不会存在数组越界的现象！！！！

- JS的数组对象

- JS数组的常用属性和方法

(2)方法: join(), 将数组元素根据指定的分隔符进行拼接, 并返回;

reverse(), 颠倒数组中元素的顺序;

注: join()不会对原先数组产生影响, 但reverse()方法会产生影响。

```
<script type="text/javascript">
    var arr = ["a","b","c","d"];
    var arrJoin = arr.join("+");
    alert(arrJoin);
    alert(arr);

    var arrReverse = arr.reverse();
    alert(arrReverse);
    alert(arr);
</script>
```

- JS的数组对象

- JS数组的常用属性和方法

(2)方法：pop()，删除并返回数组的最后一个元素；

push()，向数组的末尾添加一个或多个元素，并返回数组新长度；

注：pop()和push()均会对原先数组产生影响。

```
<script type="text/javascript">
    var arr_1 = ["a","b","c","d"];
    var arrPopElement = arr_1.pop();
    alert(arrPopElement);
    alert(arr_1);

    var arr_2 = ["a","b","c","d"];
    var newArrLength = arr_2.push("e","f","g");
    alert(newArrLength);
    alert(arr_2);
</script>
```

JS语法规则

- JS的全局函数

- JS的编码与解码

URL编码：中文与特殊字符，10进制转换成16进制，并添加%，保证数据的完整性。

<u>encodeURIComponent()</u>	把字符串编码为 URI。
<u>decodeURI()</u>	解码某个编码的 URI。

```
<head>
  <meta charset="utf-8">
  <title></title>
  <script type="text/javascript">
    var msg = "localhost:8001/html/html01.html?name=你好&password=1234";
    alert(msg);
    alert(encodeURIComponent(msg));
    alert(decodeURI(encodeURIComponent(msg)));
  </script>
</head>
```

- JS的全局函数

- URL与URI的区别

URI：统一资源标识符，标识资源的详细名称（包含资源名）；

URL：统一资源定位器，定位资源的网络位置（包含http://）。

- 资源：可以通过浏览器访问的信息统称为资源（如：图片、文本、html）

如：<http://localhost:8001/html/>

URL

</html01.html>

URI

<http://localhost:8001/html/html12.html>

URL和URI

- JS的全局函数
 - 字符串与数字的相互转换

(1) 字符串转数字

<code>parseInt()</code>	解析一个字符串并返回一个整数。
<code>parseFloat()</code> 	解析一个字符串并返回一个浮点数。

`parseInt(string);` 将字符串转换为整数，忽略小数部分（不会四舍五入）；

`parseFloat(string);` 将字符串转换为小数。

注：如果一个字符串中某一个字符无法转换成数字，那么从该字符开始停止转换，仅返回前面正确的值。（如：10.5a5 --- 10.5 `parseFloat(string)`）

如果一个字符串从第一位开始就不能转换成数字，则返回NaN（not a number，表示不是一个正确的数字）

- JS的全局函数

- 字符串与数字的相互转换

- (2) 数字转字符串

方式一：使用toString()方法；

方式二：用拼接字符串的方法实现字符串的转换；

```
<script type="text/javascript">
    var num = 10.3;
    alert(typeof(num.toString()));

    var num = 10.3;
    alert(typeof(num + "aaa"));
</script>
```

课堂练习

有5个<div>标签，每个标签中有一些文本，请利用Javascript相关对象判断哪些文本中含有连续两个或两个以上的数字，将符合条件的div中的文本进行弹窗输出。

```
<div>abdf23</div>  
<div>345t6</div>  
<div>rfgt</div>  
<div>4fo5l</div>  
<div>dwqer1</div>
```

提示：获取<div>标签中的文本

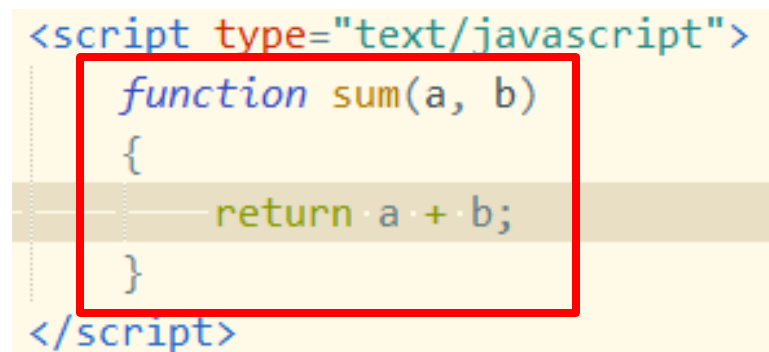
```
var arr = document.getElementsByTagName("div");  
var text = arr[i].innerHTML;
```

JS语法规则

- JS的自定义函数

- 用来将特定的功能进行封装，提高代码的复用性
- 自定义函数格式

```
function 函数名(参数列表)
{
    //函数体
}
```



```
<script type="text/javascript">
    function sum(a, b)
    {
        return a + b;
    }
</script>
```

- 定义函数时的注意事项

- (1)js的函数定义必须使用小写的function
- (2)js无需定义返回值类型，直接在function后面写函数名
- (3)参数列表不需要使用var关键字，否则报错
- (4)js函数中，return语句写不写均可，或仅仅写return;

JS语法规则

- JS的自定义函数

- js函数调用的特点

(1)js的函数一定有返回值，值与类型根据return的变量决定，如果未return具体值或不写return，则返回undefined;

```
<script type="text/javascript">
  function sum(a, b)
  {
    return a + b;
  }

  var res = sum(2, 3);
  alert(res);
</script>
```

```
<script type="text/javascript">
  function sum(a, b)
  {
    return;
  }

  var res = sum(2, 3);
  alert(res);
</script>
```

```
<script type="text/javascript">
  function sum(a, b)
  {
  }

  var res = sum(2, 3);
  alert(res);
</script>
```

JS语法规则

- JS的自定义函数

- js函数调用的特点

(2) js同名函数不存在函数重载，只有函数覆盖，后写的覆盖之前的

```
<script type="text/javascript">
  function sum(a, b, c)
  {
    alert(a + "--" + b + "--" + c);
  }
  function sum(a, b)
  {
    alert(a + "--" + b);
  }
  sum(2, 3);
</script>
```

```
function sum(a, b)
{
  alert(a + "--" + b);
}

function sum(a, b, c)
{
  alert(a + "--" + b + "--" + c);
}

sum(2, 3);
```

(3) js根据函数名称来调用，即使参数不匹配，也不影响使用

```
<script type="text/javascript">
  function sum(a, b)
  {
    alert(a + "--" + b);
  }

  sum();
</script>
```

JS语法规则

- JS的自定义对象

- 之前使用的均是js提供的对象，如何创建自定义对象？如：Student
- 自定义对象格式：函数方式

function 对象名() //定义一个对象

```
{  
  
}
```

- 创建对象：使用new关键字

var 对象变量 = new 对象名();

```
<script type="text/javascript">  
  function Student()  
  {  
    ...  
  }  
  var stu = new Student();  
  alert(stu);  
</script>
```

JS语法规则

- JS的自定义对象

- 自定义对象格式 + 添加属性与直接赋值

function 对象名() //定义一个对象

```
{  
    this.属性 = 值;  
}
```

注：this指代一个具体的对象

- 属性的访问：

- (1) 创建对象：var 对象变量 = new 对象名();

- (2) 访问属性：对象变量.属性

```
<script type="text/javascript">  
    function Student()  
    {  
        this.name = "zhangsan";  
        this.age = 20;  
    }  
    var stu = new Student();  
    alert(stu.name + ", " + stu.age);  
</script>
```

JS语法规则

- JS的自定义对象

- 自定义对象格式 + 添加属性与参数赋值

function 对象名(param_1, param_2, ...) //定义一个对象

```
{  
    this.属性1 = param_1;  
    this.属性2 = param_2;  
}
```

注：this指代一个具体的对象

```
<script type="text/javascript">  
    function Student(name, age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    var stu = new Student("lisi", 18);  
    alert(stu.name + ", " + stu.age);  
    stu.name = "wangwu";  
    stu.age = 22;  
    alert(stu.name + ", " + stu.age);  
</script>
```

- 属性的访问与重新赋值：

(1) 创建对象：var 对象变量 = new 对象名(值1, 值2);

(2) 访问属性：对象变量.属性

(3) 属性重新赋值：对象变量.属性 = 新值;

JS语法规则

- JS的自定义对象

- 在对象创建后添加属性并赋值

function 对象名(param_1, param_2, ...) //定义一个对象

```
{  
    this.属性1 = param_1;  
    this.属性2 = param_2;  
}
```

```
<script type="text/javascript">  
    function Student(name, age)  
    {  
        this.name = name;  
        this.age = age;  
    }  
    var stu = new Student("lisi", 18);  
    alert(stu.name + ", " + stu.age);  
    stu.name = "wangwu";  
    stu.age = 22;  
    alert(stu.name + ", " + stu.age);  
    stu.sex = "male";  
    alert(stu.name + ", " + stu.age + ", " + stu.sex);  
</script>
```

(1) 创建对象: var 对象变量 = new

(2) 添加属性并赋值: 对象变量.新属性 = 新属性值

JS语法规则

- JS的自定义对象

- 自定义对象格式：直接量方式

创建对象：var 对象变量 = {属性1: 值1, 属性2: 值2...};

```
<script type="text/javascript">  
  var stu = {name: "zhangsan", age: 20};  
  alert(stu);  
  alert(stu.name + ", " + stu.age);  
</script>
```

注：直接量方式创建对象时，不能再使用new关键字
给属性重新赋值，添加属性与之前一致

- 1 JS概述
- 2 JS语法规则
- 3 BOM与DOM对象**
- 4 JS事件

- 什么是BOM对象？有什么作用？
 - BOM(Browser Object Model)浏览器对象模型
 - 浏览器：IE, Chrome, Firefox等
 - 作用：用来执行浏览器的相关操作（如：浏览器地址，弹出消息等）
- 注：一般情况下，window就代表了BOM对象；
- window是js的内置对象，使用window对象调用方法时，window可以不写。如：alert(); 全称是：window.alert();

BOM对象

- BOM中window对象的弹出框
 - 警告框：alert();
 - 确认框：confirm(); confirm()有返回值，点击“确定”返回true，点击“取消”返回false。可以用在购物车“删除商品”的场景中。

```
<script>  
    //有返回值，boolean类型  
    //点击确定——> true  
    //点击取消——> false  
    var flag=confirm("是否删除");  
    alert(flag);  
</script>
```

- BOM中循环定时器

- 循环定时器：每隔多少时间，触发并执行相关功能；
- 设置循环定时器：setInterval(调用方法, 时间); 返回值为定时器Id

```
function run1() {  
    alert("我是run1");  
}  
//执行下句后，每隔1秒就会调用一次run1方法  
setInterval("run1()", 1000);
```

效果：（每隔一秒就会弹出一个对话框）

- 取消循环定时器：clearInterval(定时器Id);

```
var iid=setInterval("run1()", 2000);  
  
//clearInterval(循环定时器的ID);  
clearInterval(iid);
```

小练习：在2秒后只执行一次run1()方法。

- BOM中一次性定时器

- 一次性定时器：间隔多少时间，触发并执行相关功能，只执行一次；
- 设置一次性定时器：setTimeout(调用方法, 时间); 返回值为定时器Id

```
function run1()  
{  
    alert("你好");  
}  
var id = setTimeout("run1()", 2000);
```

- 取消一次性定时器：clearTimeout(定时器Id);

```
var id = setTimeout("run1()", 2000);  
  
clearTimeout(id);
```

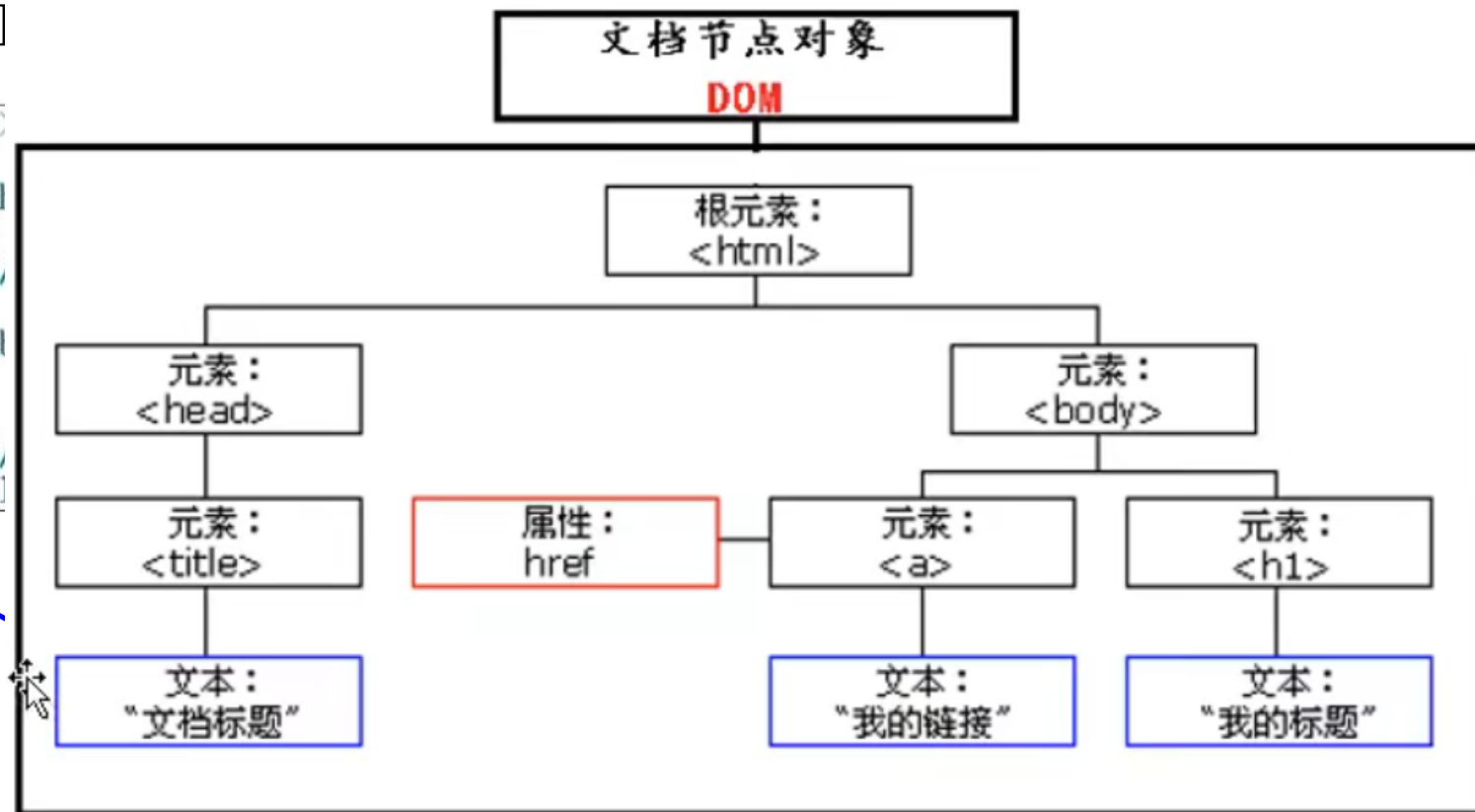
- 什么是DOM?
 - DOM（Document Object Model）：文档对象模型；
文档：html文档
 - DOM是将标记型文档中的所有内容（标签、文本、属性）都封装成对象
 - 通过操作对象的属性（对象名称、对象的值、对象的类型）与方法，来操作或改变html页面的显示效果。
 - DOM是js最关键的模型！！！！

DOM对象

- DOM

```
<html>  
<!--  
</--  
<!--  
</--  
</html>
```

一个



DOM对象

- DOM树
 - 每个标签都会被加载成DOM树上的一个元素节点对象
 - 每个标签的属性会被加载成DOM树上的一个属性节点对象；
 - 每个标签的内容体会被加载成DOM树上的一个文本节点对象；
 - 整个DOM树，是一个文档节点对象，即document(DOM)对象；
 - 一个html文档被加载到内存中，会形成一个document(DOM)对象；
- DOM树特点
 - 必定会有一个根节点（<html>节点）
 - 每个节点都是对象，节点与节点之间的关系：父子关系；
 - 文本节点（内容体）没有子节点，它是叶子节点；
 - 根节点没有父节点

DOM对象

- 获取元素节点对象（4种方式）

- getElementById(); 通过元素的ID属性获取元素节点对象;
- getElementsByName(); 通过元素name属性获取多个元素节点对象;
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象;
- getElementsByClassName(); 通过元素的class属性获取多个元素节点对象;

注：这些都是document对象的方法。

```
<input type="text" id="t1"/>

<script>
  var t1=document.getElementById("t1");
  alert(t1);
</script>
```

[object HTMLInputElement]

确定

DOM对象

- 获取元素节点对象（4种方式）

- getElementById(); 通过元素的ID属性获取元素节点对象;
- getElementsByName(); 通过元素name属性获取多个元素节点对象;
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象;
- getElementsByClassName(); 通过元素的class属性获取多个元素节点对象;

注：这些都是document对象的方法。

(1) getElementById(); 返回单个元素节点对象，如果找不到，则返回null;

```
<input type="text" id="t1"/>

<script>
    var t1=document.getElementById("t1");
    alert(t1);
</script>
```

[object HTMLInputElement]

确定

DOM对象

- 获取元素节点对象（4种方式）

- getElementById(); 通过元素的ID属性获取元素节点对象;
- getElementsByName(); 通过元素name属性获取多个元素节点对象;
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象;
- getElementsByClassName(); 通过元素的class属性获取多个元素节点对象;

注：这些都是document对象的方法。

(2) getElementsByName(); 返回元素节点对象数组，若找不到，则返回0;

```
<input type="checkbox" name="hobby" value="read"/>
<input type="checkbox" name="hobby" value="code"/>
<input type="checkbox" name="hobby" value="run"/>
```

```
//通过name属性，获取所有需要的元素
```

```
//返回是一个元素节点对象 数组
```

```
var arr=document.getElementsByName("hobby");
alert(arr.length);|
```

DOM对象

- 获取元素节点对象（4种方式）

- getElementById(); 通过元素的ID属性获取元素节点对象;
- getElementsByName(); 通过元素name属性获取多个元素节点对象;
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象;
- getElementsByClassName(); 通过元素的class属性获取多个元素节点对象;

注：这些都是document对象的方法。

(3) getElementsByTagName(); 返回元素节点对象数组，若找不到，则返回0;

```
<ul>
  <li>javaee</li>
  <li>android</li>
  <li>ios</li>
  <li>php</li>
</ul>
```

```
//通过标签名，获取所有需要的元素
var arr=document.getElementsByTagName("li");
alert(arr.length);
```

DOM对象

- 获取元素节点对象（4种方式）

- getElementById(); 通过元素的ID属性获取元素节点对象;
- getElementsByName(); 通过元素name属性获取多个元素节点对象;
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象;
- getElementsByClassName(); 通过元素的class属性获取多个元素节点对象;

注：这些都是document对象的方法。

(4) getElementsByClassName(); 返回元素节点对象数组，若找不到，则返回0;

```
<input type="radio" name="sex" value="man" class="h1"/>  
<input type="radio" name="sex" value="woman" class="h1"/>
```

```
//通过class属性值，获取所有需要的元素  
var arr=document.getElementsByClassName("h1");  
alert(arr.length);|
```

DOM对象

- 获取元素节点对象（4种方式）的注意事项

- 注意获取元素节点对象的获取位置；

- 要保证元素先被加载到内存中，才能获取到。

- 注：只要将<script>标签放在body的结束标签</body>之前，就一定没有问题

DOM对象

- DOM对象元素的常见属性

- value属性

元素对象.value, 获取元素对象的value属性值;

元素对象.value = 值; 给元素对象的value属性赋值;

```
<body>
  <input type="text" id="t1" value="你好" />
  <script>
    //需要把t1 元素value属性值, 拿出来
    var t1=document.getElementById("t1");
    alert(t1.value);

    //需要修改 value属性值      你好么
    t1.value="你好么";
  </script>
</body>
```

DOM对象

- DOM对象元素的常见属性

– class

元素

元素

注：通

```
<head>
  <meta charset="UTF-8">
  <title></title>
  <style>
    .ys1 {
      color:red;
    }
    .ys2 {
      color:blue;
    }
  </style>
</head>
<body>
  <span id="s1" class="ys1">今天天气好晴朗</span>
  <script>
    //需要把s1 元素value属性值，拿出来
    var s1=document.getElementById("s1");
    alert(s1.className);
    s1.className="ys2";
  </script>
</body>
```

DOM对象

- DOM对象元素的常见属性

- checked属性

元素对象.checked, 获取元素对象checked的属性值;

元素对象.checked = 值; 给元素对象的checked属性赋值;

注：通常用来改变元素选中的状态（单选与多选）；

元素选中：true；元素未选中：false

```
<body>
  <input type="checkbox" id="hobby" />
  <script>
    //false
    var c=document.getElementById("hobby");
    alert(c.checked);

    //想让复选框默认选中
    c.checked=true;|
  </script>
</body>
```

DOM对象

- DOM对象元素的常见属性

- innerHTML属性

元素对象.innerHTML, 获取元素对象内容体;

元素对象.innerHTML = 值; 给元素对象的内容体赋值;

注: 通常用来获取或设置元素的内容体

```
<body>
  <span id="s1">今天天气好晴朗</span>
  <script>
    //想要获取span标签的内容体
    var s1=document.getElementById("s1");
    alert(s1.innerHTML);

    //设置span 标签的内容体    今天好像要下雪
    s1.innerHTML="今天好像要下雪";

    //向span标签 追加内容体信息    , 今天0度
    s1.innerHTML+=" , 今天0度";
  </script>
</body>
```

练习（实验）

- 银行的用户须知阅读

要求：1. 在一个多行文本框<textarea>中输入若干内容；

2. 起始状态，“确认”按钮为灰色状态；

3. 用户阅读倒计时10秒，10秒后，“确认”按钮变为可点击状态。

- 1 JS概述
- 2 JS语法规则
- 3 BOM与DOM对象
- 4 JS事件**

JS事件

- JS事件是什么？有什么作用？

- 通常把鼠标或键盘的动作称为事件（Event）

如：鼠标点击、鼠标移动（移入、移除）、键盘按下等；

- 通过JS事件，我们可以完成页面的指定特效。

例：

```
<body>
  <input type = "button" value = "删除" onclick = "fun()"/>
  <script type="text/javascript">
    function fun()
    {
      confirm("确认删除吗?");
    }
  </script>
</body>
```

JS事件

- JS事件驱动机制

- JS事件的四要素：事件源、事件、监听器、绑定监听器

- (1)事件源：产生事件的组件；（按钮）

- (2)事件：由事件源所产生的动作；（鼠标点击）

- (3)监听器：专门处理 事件源 所产生的的事件；（fun()方法）

- (4)绑定监听器：让事件与监听器产生联系，如果产生了指定事件，则调用监听器处理。
onclick = “fun()”

例：

```
<body>
  <input type = "button" value = "删除" onclick = "fun()"/>
  <script type="text/javascript">
    function fun()
    {
      confirm("确认删除吗?");
    }
  </script>
</body>
```

JS事件

- 常见的JS事件

- 点击事件：onclick，由鼠标点击元素时触发。

```
<body>
  <input type="text" onclick="fun()"/>
  <script type="text/javascript">
    function fun()
    {
      alert("请输入文本!");
    }
  </script>
</body>
```

注：几乎所有的元素都有点击事件的属性！点击事件触发后，监听器执行相应的事件处理代码。

- 常见的JS事件

- 焦点事件：onfocus, onblur

焦点：整个页面的注意力，默认一个页面在同一时刻仅有一个焦点；反映了用户目前的关注点，或正在操作的组件。

例如：文本框中闪动的竖线。

- 获取焦点事件：当元素获取焦点时触发（onfocus），通常用于表单元素输入时的要求与注意事项！

- 失去焦点事件：当元素失去焦点时触发（onblur），通常用于表单中元素内容的校验！

JS事件

- 常见的JS事件

- 焦点事件：onfocus, onblur

```
<body>
  用户名1: <input type="text" onfocus="funFocus()"/>
  用户名2: <input type="text" onblur="funBlur()"/>
  <script type="text/javascript">
    function funFocus()
    {
      alert("文本框获取焦点了!");
    }
    function funBlur()
    {
      alert("文本框失去焦点了!");
    }
  </script>
</body>
```

事件处理函数（监听器）的命名建议：fun + 事件名称（去掉on）

不同浏览器之间存在差异！！！！

JS事件

- 常见的JS事件

- 域内容改变事件：onchange，元素组件的值发生变化时触发

```
<script>
    function run1() {
        alert("弹出");
    }
</script>
</head>
<body>
    <select onchange="run1()">
        <option value="beijing">北京</option>
        <option value="shanghai">上海</option>
        <option value="shenzhen">深圳</option>
    </select>
</body>
```

select选择框中默认选中的值是北京，当选择框中的值发生改变时，onchange事件会被触发

JS事件

- 常见的JS事件

- 加载完毕事件：onload，元素组件加载完毕时触发

```
<script type="text/javascript">
    function funLoad()
    {
        alert("元素加载完毕了!");
    }
</script>
</head>
<body onload="funLoad()">
    <div>安徽大学</div>
    <div>互联网学院</div>
    <div>互联网金融系</div>
    <div>19级互联网金融</div>
</body>
```

js代码放在body的结束标签之前，保证元素先加载；但在<body>添加了onload事件后，js代码就可以放在任意位置；

JS事件

- 常见的JS事件

- 表单提交事件: `onsubmit`, 表单的提交按钮被点击时触发

```
<body>
  <form onsubmit="return funSubmit()" method="get">
    用户名: <input type="text" name="username"/><br/>
    <input type="submit" value="提交" />
  </form>
  <script type="text/javascript">
    function funSubmit()
    {
      return confirm("确认提交吗?");
    }
  </script>
</body>
```

注: 该事件需要在处理函数（监听器）中返回boolean类型的值来执行 提交/阻止(true/false) 表单提交的操作。常用于表单校验！

JS事件

- 常见的JS事件

```
<style type="text/css">
  .imgclass
  {
  }
  .imgclass1
  {
    width: 320px;
    height: 400px;
  }
</style>
```

鼠标移入事件（onmouseover），

鼠标移出事件（onmouseout）

当鼠标移入到元素组件时触发；（图片样式改变）

当鼠标移出到元素组件时触发；

```
<body>

<script type="text/javascript">
  function funMouseOver()
  {
    var img = document.getElementById("imgId");
    img.className = "imgclass1";
  }
  function funMouseOut()
  {
    var img = document.getElementById("imgId");
    img.className = "imgclass";
  }
</script>
</body>
```

JS事件

- JS事件的两种绑定方式

- (1)元素事件句柄绑定（一个事件绑定一个处理函数）

将事件以元素属性的方式写到标签的内部，进而绑定对应的处理函数；

```
<input type="text" value="111" onclick="funClick1()"/>
<script type="text/javascript">
    function funClick1()
    {
        alert("run1");
    }
</script>
<input type="text" value="222" onclick="funClick2('你好')"/>
<script type="text/javascript">
    function funClick2(str)
    {
        alert(str);
    }
</script>
<input type="text" value="333" onclick="funClick3(this)"/>
<script type="text/javascript">
    function funClick3(obj)
    {
        alert(obj.value);
    }
</script>
```

绑定无参函数

绑定基本数据类型
为参数的函数

绑定引用数据类型
为参数的函数

JS事件

- JS事件的两种绑定方式

- (1)元素事件句柄绑定（一个事件绑定多个处理函数）

```
<body>
  <input type="text" value="111" onclick="funClick1(),funClick2('你好'),funClick3(this)"/>
  <script type="text/javascript">
    function funClick1()
    {
      alert("run1");
    }
    function funClick2(str)
    {
      alert(str);
    }
    function funClick3(obj)
    {
      alert(obj.value);
    }
  </script>
</body>
```

格式：将事件以元素属性的方式写到标签的内部，多个处理函数之间用逗号隔开。

JS事件

- JS事件的两种绑定方式

- (2)DOM绑定：使用DOM的属性进行绑定

写法步骤：①获取事件源的dom对象；

②编写监听器（事件处理函数）

③对象.事件属性 = 监听器名称（事件处理函数的名称）

```
<body>
  <input type="button" id="btnId" value="点击"/>
  <script type="text/javascript">
    var btn = document.getElementById("btnId");
    btn.onclick = funClick;
    function funClick()
    {
      alert("我被点击了!");
    }
  </script>
</body>
```

注：此时，监听器名称不带小括号！！但该写法只能绑定一个事件处理函数，且是无参的。

JS事件

- JS事件的两种绑定方式

- (2)DOM绑定：使用DOM的属性进行绑定

写法二步骤：①获取事件源的dom对象

②编写监听器（事件处理函数）

③对象.事件属性 = 匿名事件处理函数

```
<input type="button" id="btnId" value="点击"/>
<script type="text/javascript">
    var btn = document.getElementById("btnId");
    function funClick1()
    {
        alert("我被点击了!");
    }
    function funClick2(str)
    {
        alert(str);
    }
    btn.onclick = function(){
        funClick1();
        funClick2("你好");
    }
</script>
```

注：该写法可以绑定多个事件处理函数，函数有参无参均可。建议使用！

- JS事件的两种绑定方式
 - 元素句柄绑定 VS DOM绑定

绑定方式	优点	缺点
元素句柄绑定	开发便捷、传参方便、可以绑定多个函数	html代码与JS代码糅合在一起，不利于多部门联合开发后期维护
DOM绑定	html代码与JS代码相分离，便于多部门联合开发后期维护	匿名函数的写法对初学者不太友好

课堂练习

用户名	
密码	
性别	☒ 男 ☐ 女
喜好	☐ 编程 ☐ 跑步 ☐ 游泳
头像	浏览... 未选择文件。
所在城市	北京 ▼
用户类别	普通用户
用户简介	
注册	重置

在表单的基础上添加特效：

1. 鼠标获取密码框焦点时，提示密码输入要求：8位数字，首位不能为0.
2. 鼠标失去密码框焦点时，进行密码校验；不满足要求的弹框提示；
3. 鼠标点击注册按钮时，进行表单校验，所有内容不能为空，否则不能提交。
当发现某些元素值为空时，进行弹窗提醒！

谢谢!

Thanks for your attention!